

AutoPen: Towards Autonomous Penetration Testing Using LLM-Powered Agents

Jiahao Mei
CENI HeFei

University of Science and Technology of China
HeFei, China
mjh1@mail.ustc.edu.cn

Yuanyi Ma
CENI HeFei

University of Science and Technology of China
HeFei, China
yym672@mail.ustc.edu.cn

Shuangwu Chen*

CENI HeFei
University of Science and Technology of China
HeFei, China
chensw@ustc.edu.cn

Huizi Song
CENI HeFei

University of Science and Technology of China
HeFei, China
sherlockzero@mail.ustc.edu.cn

Abstract

Penetration testing is an effective means of ensuring system security, yet it suffers from drawbacks such as being time-consuming, labor-intensive, and requiring highly specialized operators. The advancement of large language models (LLMs) has made it feasible to utilize them for automated penetration testing. In this paper, we propose AutoPen, a method designed to autonomously conduct penetration testing without human intervention. AutoPen employs techniques such as Retrieval-Augmented Generation (RAG), chain-of-thought reasoning, and structured output to obtain more relevant and accurate responses from the LLM. Furthermore, it incorporates a carefully designed penetration testing workflow with task decomposition and specialized agents to ensure efficient and autonomous operation. We conducted penetration testing experiments on various vulnerable hosts from VulnHub. The results demonstrate that AutoPen performs automated penetration testing with high stability and exhibits significantly superior capability compared to open-source methods and other approaches described in related literature.

CCS Concepts

• **Computing methodologies** → **Multi-agent systems**; • **Security and privacy** → **Penetration testing**; • **Networks** → *Network security*.

Keywords

Large language models, LLM, RAG, LLM agent, penetration testing

ACM Reference Format:

Jiahao Mei, Shuangwu Chen, Yuanyi Ma, and Huizi Song. 2025. AutoPen: Towards Autonomous Penetration Testing Using LLM-Powered Agents. In *The 9th International Conference on Computer Science and Application Engineering (CSAE 2025)*, October 19–21, 2025, Shanghai, China. ACM, New York, NY, USA, 6 pages. <https://doi.org/10.1145/3772886.3772899>



This work is licensed under a Creative Commons Attribution 4.0 International License. *CSAE 2025, Shanghai, China*

© 2025 Copyright held by the owner/author(s).
ACM ISBN 979-8-4007-1326-2/25/10
<https://doi.org/10.1145/3772886.3772899>

1 Introduction

Penetration testing, sometimes called penetration testing or ethical hacking[1], is a critical technique for countering cyber threats, aimed at preventing attacks by identifying security vulnerabilities within a system. Traditionally, penetration testing is performed manually by skilled security experts. The conventional process involves scanning target systems, synthesizing information to identify entry points, testing for vulnerabilities, and reporting the final results[2]. While penetration testing significantly enhances system security, the manual execution of this process demands substantial labor, time, and extensive expertise and practical experience from the operator.

Researchers have never ceased their exploration of fully automated penetration testing that operates without human intervention. Early researchers assumed that defenders possessed complete observability and modeled attack planning as an attack graph problem. However, such methods lacked the flexible adaptability required in real-world environments. Subsequent studies introduced uncertainty, modeling attack planning as a Markov Decision Process (MDP) and employing various reinforcement learning algorithms to identify optimal attack paths[4]. For instance, Yousefi et al. utilized the MulVaL tool to generate attack graphs and transition matrices, which were combined with the Q-learning algorithm to produce attack paths with maximum reward values[5]. As an extension of the MDP approach, researchers adopted the Partially Observable Markov Decision Process (POMDP)[6] and reinforcement learning algorithms[7, 8] to further address uncertainties in the environment and action outcomes. These methods better reflect the realistic scenario where attackers have limited knowledge of the target system. Nevertheless, most of these studies remain largely theoretical and lack the capability for practical deployment and application.

Recent significant advancements in Large Language Models (LLMs) have led to rapid improvements in their generative capabilities. General-purpose LLMs such as ChatGPT[9] and Deepseek[10, 11], are demonstrating robust performance and introducing new transformations and opportunities across various fields. This progress also makes automated penetration testing utilizing LLMs a possibility. To address the inherent limitations of general-purpose LLMs, existing methods employ multi-agent designs based on role-playing,

dividing the entire penetration testing process into a series of sub-tasks handled by specialized agents. By standardizing prompt outputs, these approaches guide LLMs to generate precisely required content and avoid overly long dialogues, thereby making effective use of historical information[12]. Building on this foundation, subsequent research has introduced the first open benchmark for LLM-based automated penetration testing [13] and proposed using Retrieval-Augmented Generation (RAG) to provide external knowledge for fine-tuning LLMs, thereby enhancing their domain-specific expertise [14]. Nevertheless, these methods still cannot eliminate the reliance on human expertise, requiring manual intervention for strategic decision-making and specific implementation. Consequently, the proficiency of the operator significantly influences the testing progression and final outcomes.

Therefore, we propose AutoPen, an attempt to construct a method capable of autonomously executing penetration testing without human intervention. In this work, we design an LLM-enhanced penetration testing framework based on a multi-agent architecture and introduce a novel partitioning of the penetration testing workflow to ensure its autonomous operation. Functions as orchestrating the process, logging activities, handling error correction and rollback are realized by LLM agents, which helps process operation. By organizing information into a knowledge graph, we facilitate the automated penetration testing process, while critical issues are addressed through multi-agent integrated responses to mitigate hallucinations. Finally, we implement the proposed method and construct experiments to validate its effectiveness. The main contributions of this paper are as follows.

- To address the issue of low automation levels and heavy reliance on human expertise in existing penetration testing methods, we partition the penetration testing workflow into distinct stages, each managed by a specialized agent. Furthermore, we meticulously design Inspection Agent to orchestrate the process and ensure robust operation. Information is organized into a knowledge graph for efficient access, guaranteeing that the penetration testing can be completed autonomously without external human intervention.
- To overcome the limitations of general-purpose LLMs—such as limited penetration testing knowledge, susceptibility to infinite loops due to ambiguous instructions or formats, and tendencies towards hallucination, which hinder their ability to successfully complete tests—we employ techniques including Retrieval-Augmented Generation, chain-of-thought reasoning, and structured output to enhance the LLMs’ generative performance. We also propose a multi-agent integration approach to mitigate generation errors, ensuring in-depth testing of the target host can be conducted.
- We implemented the proposed method and constructed a specific experimental environment to validate its effectiveness. The results demonstrate that AutoPen can effectively complete penetration testing tasks with high stability, exhibiting superior performance compared to similar methods.

2 METHODOLOGY

2.1 System Overview

Given the limited context window of large language models, prolonged operation would lead to the loss of early-stage information, making it infeasible to complete the entire penetration testing process within a single session. Adopting a role-based agent design approach, we established three distinct modules within the large model framework: a Scanning Agent (SA) for environment reconnaissance, an Analysis Agent (AA) for vulnerability analysis, and an Execution Agent (EA) for vulnerability exploitation. By decomposing the penetration testing workflow into a series of subtasks assigned to these specialized agents, we significantly reduce the context length required for each agent’s large model. This approach also enhances the model’s focus and performance by assigning more specific and well-defined tasks. Furthermore, given the high interdependence between different stages of penetration testing, it is crucial to summarize the information and conclusions generated by the large model and pass them to subsequent tasks. To address this, we designed an Inspection Agent (IA) responsible for self-inspection and backtracking, which automates this information handover. This design ensures minimal loss of critical information while maintaining a clear focus at each stage.

Unlike the other agents, the IA is not responsible for generating or executing specific commands. Instead, it serves three primary functions: orchestration, self-inspection, and backtracking. Orchestration involves managing the automated execution of the overall workflow from a macro perspective, accurately transmitting information between different modules, and generating the final penetration test report by retrieving historical logs upon completion. Self-inspection refers to promptly detecting errors in other modules, halting their execution, and attempting to diagnose and rectify the issue based on the context and error messages. Backtracking is initiated when the process cannot proceed; the agent then rolls back operations to a previous stable state and attempts to explore alternative paths.

The specific operational workflow of these agents is illustrated in Fig. 1. The functions of each agent and the role of the IA within each stage will be detailed in Sections 2.B to 2.D, corresponding to the different phases of the penetration test.

2.2 Environmental Detection

The primary objective of the initial phase of penetration testing is to comprehensively gather information about the target system, such as all open ports, running services, and their versions. In this phase, the IA specifies the target IP address to the SA. The SA then enters a self-iterative loop, autonomously generating and executing reconnaissance commands. The results of each command execution are returned to the SA, which analyzes the outcomes, summarizes the information, and subsequently generates and executes the next scanning step until the process is complete. Upon completion of the scan, the IA receives a results report—including open ports, running services, and their versions—along with a log file documenting the entire scanning process, both of which are utilized in subsequent phases.

To prevent task failures caused by non-standardized responses, this study meticulously designs textual task instructions provided

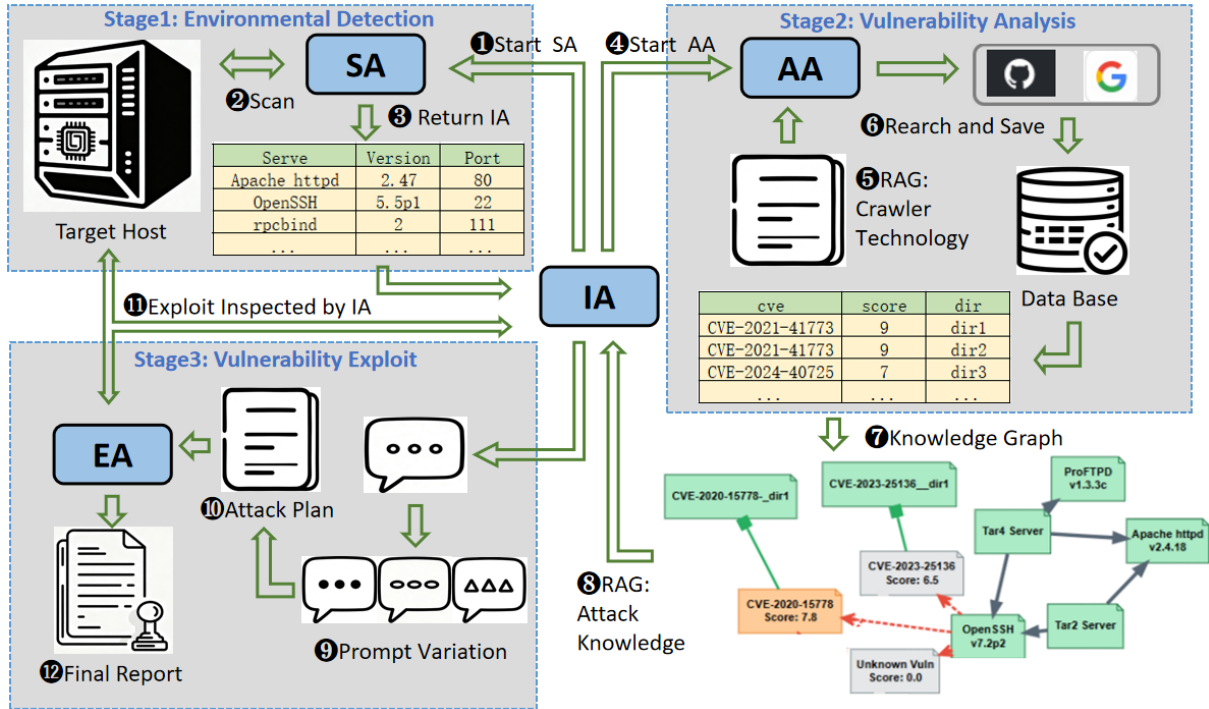


Figure 1: Overview of the proposed AutoPen.

to the agents throughout the system workflow. These instructions include detailed descriptions of the task and role, alongside bypassing the safety policies. For each specific task, the chain-of-thought technique is employed to guide the large language model in following logical steps to solve problems, significantly enhancing its reasoning capability for complex tasks. For particular tasks, structured output techniques are utilized by specifying the output format and providing comprehensive examples, directing the model to generate outputs in a fixed schema. This ensures seamless integration with subsequent tasks and execution. These precise instructions collectively help mitigate hallucinations in the large language model.

2.3 Vulnerability Analysis

Upon receiving the target system information provided by the IA, the AA is responsible for performing vulnerability analysis. To address the limitation of insufficient penetration testing knowledge, we employ Retrieval-Augmented Generation technology, constructing a knowledge base by extracting key information from relevant sources to enhance the large language model’s performance. Since direct online searches often yield redundant and irrelevant information, resulting in low efficiency, we utilize the RAG technique to extract crucial details from the raw search results. RAG combines information retrieval with large language models by fetching relevant information from an external knowledge base and using it as contextual prompts for the model, thereby enhancing its reasoning and generative capabilities for complex, domain-specific problems. The AA compiles and summarizes the search results, retrieves relevant segments as instructed (such as applicable versions and execution

prerequisites), extracts key information, and ultimately constructs a concise vulnerability exploitation database.

During runtime, the Vulnerability Analysis Agent performs two rounds of online searches based on the services and applications running on the open ports of the target system to gather information on potential vulnerabilities. The objective of the first search is to query search engines and public vulnerability databases to identify potential attack surfaces of the target system. The second search leverages the potential attack surfaces identified in the first round to further search for open-source vulnerability exploitation code, tutorials, and other relevant resources, thereby obtaining program-level attack knowledge. These retrieved documents are stored locally for reference in subsequent penetration testing stages.

Considering that standard Retrieval-Augmented Generation struggles to capture complex semantic relationships within documents, we pre-process the documents by organizing them into a Knowledge Graph (KG) to facilitate efficient retrieval and reasoning by the large language model. After the vulnerability exploitation database is constructed, we further utilize it to build a Knowledge Graph. The knowledge graph is formally defined as a directed graph $G=(V,E)$, where: V represents the set of nodes, which includes categories such as system nodes, software nodes, vulnerability nodes, and resource nodes; E represents the set of directed edges, describing relationships between nodes, e.g., “runs”, “has_vuln”, and “has_res”. The detailed definitions are as follows:

The knowledge graph is formally defined as a directed graph $G = (V, E)$, where V represents the set of nodes, which includes categories such as system nodes, software nodes, vulnerability nodes, and resource nodes; E represents the set of directed edges,

describing relationships between nodes, e.g., *runs*, *has_vuln*, and *has_res*.

The detailed definitions are as follows:

Node Sets

- **Set of system nodes:** $V_s = \{s \mid s \in S\}$, where S is the set of all system names (obtained from the keys in the first part of the document);
- **Set of software nodes:** $V_w = \{(w, v) \mid w \in W, v \in V_w\}$, where W is the set of software names and V_w is the set of versions; each software node is uniquely identified by a software name and a version;
- **Set of vulnerability nodes:** $V_v = \{(c, sc) \mid c \in C, sc \in \mathbb{R}\}$, where C is the set of CVE IDs, and sc denotes the vulnerability score;
- **Set of resource nodes:** $V_r = \{(n, p) \mid n \in N, p \in P\}$, where N is the set of resource names and P is the set of paths.

Edge Sets

- **Runs relation:** $E_{runs} \subseteq V_s \times V_w$, representing the relationship between a system node and a software node that it runs;
- **Has_vuln relation:** $E_{has_vuln} \subseteq V_w \times V_v$, indicating that a software node is associated with a vulnerability node;
- **Has_res relation:** $E_{has_res} \subseteq V_v \times V_r$, denoting that a vulnerability node is linked to a relevant resource node.

Edge Set Definitions

The edge sets are explicitly constructed as follows:

$$E_{runs} = \bigcup_{s \in S} \{(s, (w, ver)) : w \in C(s), ver = C(s)[w][0]\} \quad (1)$$

$$E_{has_vuln} = \bigcup_{s \in S} \{(h(id_str), (cve_id, score))\} \quad (2)$$

$$E_{has_res} = \bigcup_{s \in S} \{((cve_id, score), (name_res, path))\} \quad (3)$$

where:

- S denotes the set of all systems
- $C(s)$ represents software installed on system s
- $L(s)$ indicates the vulnerability list associated with system s
- $h(\cdot)$ is a hashing function for identifier strings
- For E_{has_vuln} : $id_str, cve_id, score \in L(s)$
- For E_{has_res} : $cve_id, score, name_res, path \in L(s)$

For instance, regarding the target host Tar1 Server, after detecting the open service Apache httpd vNone in the previous step, edges are constructed according to Equation (1). Subsequently, associated CVEs and their threat levels (e.g., CVE-2021-42013, Score 8.0) are retrieved via cvemap, and corresponding edges are established using Equation (2). Finally, relevant documentation (e.g., CVE-2021-4201-httpd-dir1) is retrieved and downloaded online, with edges constructed based on Equation (3). The collective data from various range machines ultimately forms a complex network structure. The resulting knowledge graph returned to the Inspection Agent is illustrated in the figure below. We represent hosts, services, and related documentation as green nodes, while CVEs are represented

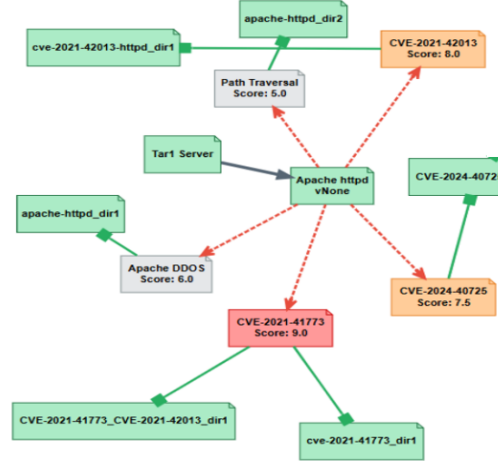


Figure 2: Example of Knowledge Graph.

as nodes colored according to their severity scores (red, orange, gray). The resulting knowledge graph is illustrated as Fig. 2.

2.4 Vulnerability Exploit

On obtaining the knowledge graph, the IA initiates the process by analyzing the potential attack surfaces. It references service/application versions and vulnerability types to formulate a penetration testing plan aimed at gaining and escalating access to the target system. This plan comprises a series of exploitable vulnerabilities, sorted by confidence level, along with references to their associated locations of documentation. The IA then sequentially invokes the EA to test each vulnerability. The role of the EA is to generate and execute a set of penetration testing actions constituting a single attack plan, with the objective of compromising the target system. Agents first utilizes RAG to access local documentation, thereby enhancing its expertise and performance. It then generates detailed steps for vulnerability exploitation and automatically executes them until success is achieved or the plan is completed.

After all individual attack plans are executed, the IA synthesizes all the returned results into a comprehensive document. This document is provided to the EA with instructions for a final penetration attempt. During this attempt, the EA is required to exercise creative, expert-like judgment based on the provided document, striving to uncover security vulnerabilities in the target system as thoroughly and deeply as possible. The outcome of this final attempt serves as the primary basis for the ultimate penetration test report.

When the IA generates an attack plan, the lack of complete information necessitates that LLMs supplement certain details to finalize the plan's generation. To mitigate potential hallucinations introduced by the LLM during this process, we designed a Multi-Agent Integration approach. This involves initiating multiple agents to generate responses for complex tasks, followed by using the LLM itself to review and eliminate errors resulting. In the described workflow, operational errors generated by the specific task agents—SA, AA, and EA—are corrected by the IA. However, errors originating from the IA itself pose a significant challenge: designing an

additional dedicated error-correction module could itself malfunction, potentially leading to an infinite loop of error correction. To minimize the possibility of the IA producing erroneous results on critical issues, we integrated the Multi-Agent approach directly into its reflection process to enhance reliability. Specifically, we initiate a separate LLM session dedicated to “prompt variation.” This session mutates the original input prompt into multiple versions that are semantically equivalent but phrased differently. These varied prompts are then dispatched simultaneously to multiple independent sessions. The final response is derived by synthesizing the answers from all these sessions. This method of generating and consolidating diverse responses helps to average out inconsistencies and mitigate the impact of hallucinations from any single LLM instance.

The Execution Agent exhibits a high error rate when attempting to exploit a single vulnerability outlined in the attack plan. This is partly due to its susceptibility to entering infinite Q&A loops caused by formatting issues or interpretational discrepancies, and partly because an error in any single step can lead to the complete failure of the whole vulnerability exploitation attempt. To address this, we enhanced the IA with an inspection and backtracking capability. This functionality allows the IA to review the failure logs returned by the EA. If the review confirms the step was executed correctly but simply failed (e.g., the vulnerability was not present or exploitable), the IA proceeds with the overall attack plan. However, if the review identifies an execution error, the IA generates a corrective solution and feeds it back to the EA to rectify the mistake before reattempting the step.

3 EVALUATION

A well-designed benchmark dataset should encompass tasks of varying difficulty levels to thoroughly evaluate automated penetration testing methods. Therefore, we selected VulHub as our benchmark dataset. VulHub is an open-source collection containing numerous vulnerable Docker environments, allowing for quick assessment of environmental attributes and exploitation difficulty via relevant metrics, making it an ideal benchmark dataset for penetration testing. We constructed a benchmark comprising 10 penetration testing targets to ensure a comprehensive evaluation. The proposed method, acting as the attacker, was deployed on a virtual machine running Ubuntu 22.04 LTS. It was provided with the IP addresses of target hosts running on separate virtual machines to conduct penetration tests. The attacker machine was equipped with common tools and maintained network connectivity with the victim machines via NAT, simulating real-world network conditions while the attacker attempted to exploit vulnerabilities. We used PentestAgent and Nessus as baseline methods for comparison. PentestAgent is an LLM-based automated penetration testing system that integrates a multi-agent design and Retrieval-Augmented Generation technology. It enhances penetration testing knowledge and automates various tasks, capable of operating with minimal human intervention. Nessus is a widely used automated vulnerability scanning tool with over 210,000 scanning plugins covering approximately 85,000 vulnerabilities, enabling effective penetration testing[3]. Due to space limitations, the detailed reports generated by the methods cannot be fully presented here. We evaluated the

experimental results using stage-wise completion rate as the metric, specifically examining each method’s ability to: successfully scan target port services(Scan), complete vulnerability exploitation attempts(Analysis), and identify the number of actually exploitable vulnerabilities(Exploit). The experimental results are summarized as Fig. 3.

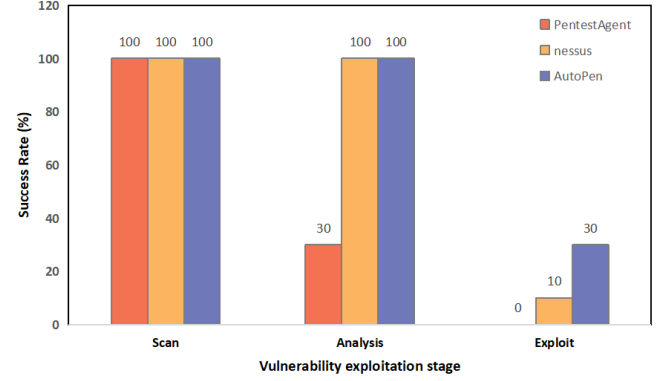


Figure 3: Stage-wise Completion Rate.

The results demonstrate that the proposed method performs effectively across all stages. PentestAgent often fails to collect target information due to anti-crawler mechanisms on websites and underutilizes locally downloaded data. In contrast, our approach equips the Vulnerability Analysis Agent with local technical documentation on bypassing anti-crawler measures, thereby improving the success rate of data retrieval. While Nessus successfully conducts scans and provides detailed reports, its output is primarily intended for user awareness. For instance, Nessus frequently flags issues such as outdated Ubuntu versions recommending upgrades or ICMP timestamp responses potentially enabling attacks—findings that are often impractical for actual penetration testing. In comparison, the proposed method achieves a higher success rate in returning actionable and practically exploitable results.

We further analyzed the quantity of data collected by PentestAgent and AutoPen for each target host. The results, shown in Fig. 4, indicate that AutoPen retrieves necessary information with superior stability compared to PentestAgent. Although AutoPen downloads a lower total volume of data due to stricter filtering criteria, it more effectively completes the final penetration testing tasks. This proves that our method reliably performs vulnerability information retrieval and analysis, fully leverages the generative capabilities of large language models, and achieves stronger vulnerability detection with relatively less external data.

We conducted an ablation study on the proposed method itself to analyze the effectiveness of the Inspection Agent. Specifically, we disabled the functionalities related to the IA and ran the process manually without reasoning capabilities. The method without the IA is denoted as AutoPe_noIA. The results indicate that while AutoPen_noIA could successfully complete the environment reconnaissance and vulnerability analysis tasks, it only managed to identify one supposedly exploitable vulnerability. Furthermore, upon manual investigation, we found that the success report returned by AutoPen_noIA claimed a successful DDoS attack—asserting that the

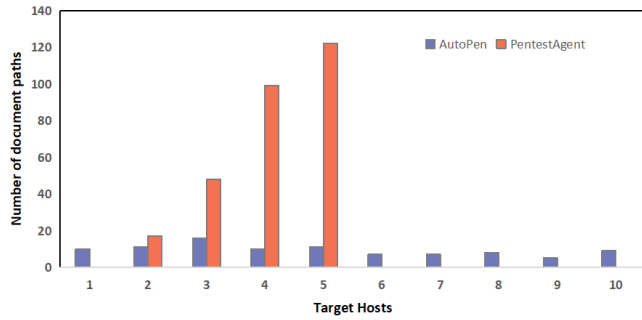


Figure 4: Collected Documents Numbers.

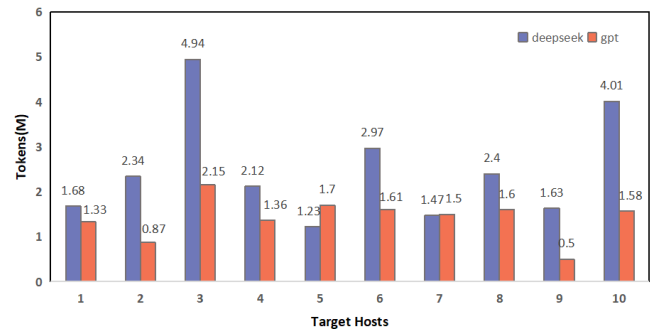


Figure 5: Cost of Different LLMs.

Table 1: Ablation Experiment And Different LLMs Results

Target Host	AutoPen_noIA	DeepSeek	GPT
Target Host1	×	✓	×
Target Host2	×	×	×
Target Host3	×	×	×
Target Host4	×	×	×
Target Host5	×	✓	×
Target Host6	×	×	×
Target Host7	×	×	×
Target Host8	×	×	×
Target Host9	×	×	×
Target Host10	×	✓	✓

target server was rendered unresponsive due to resource exhaustion—based on the rationale that port 80 was open and reachable via ICMP ping, yet the server did not respond to any HTTP connection attempts (including basic requests, specific file access, and extended time-out requests). Our manual analysis revealed that the target server was inherently configured not to respond to such connection attempts. Therefore, the success report was invalid, and AutoPen_noIA actually failed to discover any genuinely exploitable vulnerabilities. This result confirms that the proposed Inspection Agent significantly enhances the penetration testing capability of the system.

In our methodological pipeline, we intentionally decoupled the underlying large language model from specific functions and workflow processes. This design enables our approach to be flexibly migrated across different LLMs. We conducted vulnerability exploitation tests using GPT-4o and DeepSeek-V3.1 respectively; the results and associated costs are summarized in the table below. As can be observed, DeepSeek consumed relatively more tokens while achieving superior results. Given that the code and runtime language are identical, the performance difference may be attributed to a higher compatibility between our method and DeepSeek’s operational style. Despite these specific performance variations, both models were capable of conducting in-depth testing on the target hosts, demonstrating the independence of our proposed method from the underlying LLM.

References

- [1] Bishop M. About penetration testing[J]. IEEE Security & Privacy, 2007, 5(6): 84-87.
- [2] Denis M, Zena C, Hayajneh T. Penetration testing: Concepts, attack methods, and defense strategies[C]//2016 IEEE Long Island Systems, Applications and Technology Conference (LISAT). IEEE, 2016: 1-6.
- [3] Al Shebli H M Z, Beheshti B D. A study on penetration testing process and tools[C]//2018 IEEE Long Island Systems, Applications and Technology Conference (LISAT). IEEE, 2018: 1-7.
- [4] Schwartz J, Kurniawati H, El-Mahassni E (2020) POMDP + information-decay: incorporating defender’s behaviour in autonomous penetration testing. In: ICAPS, pp 235–243
- [5] Yousef M, Mtetwa N, Zhang Y, Tianfeld H (2018) A reinforcement learning approach for attack graph analysis. In: TrustCom/BigDataSE, pp 212–217
- [6] Carlos Sarraute, Olivier Buffet, and Jörg Hoffmann. 2012. POMDPs make better hackers: Accounting for uncertainty in penetration testing. In Proceedings of the AAAI Conference on Artificial Intelligence, Vol. 26. 1816–1824.
- [7] Jinyin Chen, Shulong Hu, Haibin Zheng, Changyou Xing, and Guomin Zhang. 2023. GAIL-PT: An intelligent penetration testing framework with generative adversarial imitation learning. Computers & Security 126 (2023), 103055.
- [8] Zhenguo Hu, Razvan Beuran, and Yasuo Tan. 2020. Automated penetration testing using deep reinforcement learning. In 2020 IEEE European Symposium on Security and Privacy Workshops (EuroS&PW). IEEE, 2–10.
- [9] Brown T, Mann B, Ryder N, et al. Language models are few-shot learners[J]. Advances in neural information processing systems, 2020, 33: 1877-1901.
- [10] Bi X, Chen D, Chen G, et al. Deepseek llm: Scaling open-source language models with longtermism[J]. arXiv preprint arXiv:2401.02954, 2024.
- [11] Liu A, Feng B, Xue B, et al. Deepseek-v3 technical report[J]. arXiv preprint arXiv:2412.19437, 2024.
- [12] Gelei Deng and Yi Liu and V, et al. PENTESTGPT: Evaluating and Harnessing Large Language Models for Automated Penetration Testing[C]. 33rd USENIX Security Symposium (USENIX Security 24), pp. 847–864. Jun 2024.
- [13] Shen X, Wang L, Li Z, et al. Pentestagent: Incorporating llm agents to automated penetration testing[C]//Proceedings of the 20th ACM Asia Conference on Computer and Communications Security. 2025: 375-391.
- [14] Isozaki I, Shrestha M, Console R, et al. Towards automated penetration testing: Introducing llm benchmark, analysis, and improvements[C]//Adjunct Proceedings of the 33rd ACM Conference on User Modeling, Adaptation and Personalization. 404-419. Feb 2025.